Poisoning the Well: Adversarial Poisoning on ML-Based Software-Defined Network Intrusion Detection Systems

Tapadhir Das, *Member, IEEE*, Raj Mani Shukla, and Shamik Sengupta, *Senior Member, IEEE*

***Abstract*—With the usage of Machine Learning (ML) algorithms in modern-day Network Intrusion Detection Systems (NIDS), contemporary network communications are efficiently protected from cyber threats. However, these ML algorithms are starting to be compromised by adversarial attacks that ambush the ML pipeline. This paper demonstrates the feasibility of an adversarial attack called the Cosine Similarity Label Manipulation (CSLM) which is geared toward compromising training labels for ML-based NIDS. The paper develops two versions of CSLM attacks: Minimum CSLM (Min-CSLM) and Maximum CSLM (Max-CSLM). We demonstrate the attacks' efficacy towards single and multi-controller Software-defined Network (SDN) setups. Results indicate that the proposed attacks provide substantial deterioration of classifier performance in single SDNs, specifically, those that utilize Random Forests (RF), which deteriorate ≈50% under Min-CSLM attacks, and Support Vector Machines (SVM), which undergo ≈60% deterioration from a Max-CSLM attack. We also note that RF, SVM, and Multi-layer Perceptron (MLP) classifiers are also extensively vulnerable to these attacks in Multi-controller SDN setups (MSDN) as they incur the most observed utility deterioration. MLP-based uniform MSDNs incur the most deterioration under both proposed CSLM attacks with ≈28% decrease in performance, while SVM and RF-based variable MSDNs incur the most deterioration under both CSLM attacks with ≈30% and ≈ 35% decrease in performance, respectively.**

***Index Terms*—Adversarial attacks, data poisoning, network intrusion detection, software-defined networks, machine learning.**

I. Introduction

THE increased usage of the internet and its connected applications have taken communications to unprecedented heights, and because of this, software-defined networks (SDN) have turned into an indefinite component of these communication capabilities. SDNs decouple the control and data planes, and a logically centralized SDN controller administers the wide-scale network of switches. The SDN controller is primarily responsible for injecting flow tables within the network switches. This helps control network operations and makes it easier to perform any network alterations, upgradations, and monitoring capabilities [1]. Due to their consolidated architecture, performing data mining and analysis for improved efficiency and security is uncomplicated [2]. SDN-based networks form integral communication components of industrial organizations like energy and healthcare sectors, military defense, intelligent traffic control, and electrical grids.

With the rise in SDN use in industry, comes the increased risk of cyber attacks like network intrusions which aim to compromise the four major principles of computing systems: availability, authority, confidentiality, and integrity. These network intrusions can adversely affect all layers in the SDN architecture. These threats are difficult to detect unaided as they tend to display similar network patterns as that of normal network traffic. This has led to the rise in network intrusion detection systems (NIDS) whose goal is to protect by actively monitoring the network traffic patterns for any form of malicious behavior [3].

The usage of machine learning (ML) for NIDS has gained attention as a prominent method, due to its ability to find correlations in traffic flow patterns. This is advantageous as ML algorithms can perform automated discernment of malicious network patterns from benign scenarios. ML methods can be simply trained, tested, and deployed in the SDN controller due to possessing sufficient computational resources [4]. However, the increased usage of ML for NIDS has made the security of these algorithms a prominent issue [5] [6] as they can be vulnerable to adversarial attacks. Adversarial attacks are cyber attacks that aim to corrupt the functionality of ML algorithms by performing adversarial manipulations on them. These attacks aim to manipulate training data and model sensitivities to adversely affect the performance of the classifier.

A subset of adversarial attacks against ML algorithms is called poisoning attacks, where the goal is to tamper with a target model or data that makes the algorithm provide less optimal performance and triggers misclassifications [7]. Common poisoning attacks include data injection, logic corruption, and data manipulation attacks like label manipulation. Label manipulation attacks are data poisoning attacks where the labels of the training data can be adversarially perturbed to decrease the performance of the trained classifier. An illustration of a label manipulation attack on an ML system can be seen in

Received 23 September 2022; revised 17 June 2024; accepted 1 November 2024. Date of publication 5 November 2024; date of current version 25 December 2024. This work was supported by the National Science Foundation (NSF) under Award 2019164. Recommended for acceptance by Dr. Pan Zhou. *(Corresponding author: Tapadhir Das.)*

Tapadhir Das is with the Department of Computer Science, University of the Pacific, Stockton, CA 95211 USA (e-mail: tdas@pacific.edu).

Raj Mani Shukla is with the School of Computing and Information Science, Anglia Ruskin University, CB1 1PT Cambridge, U.K. (e-mail: raj.shukla@aru.ac.uk).

Shamik Sengupta is with the Department of Computer Science and Engineering, University of Nevada, Reno, NV 89557 USA (e-mail: ssengupta@unr.edu).

Digital Object Identifier 10.1109/TNSE.2024.3492032

2327-4697 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

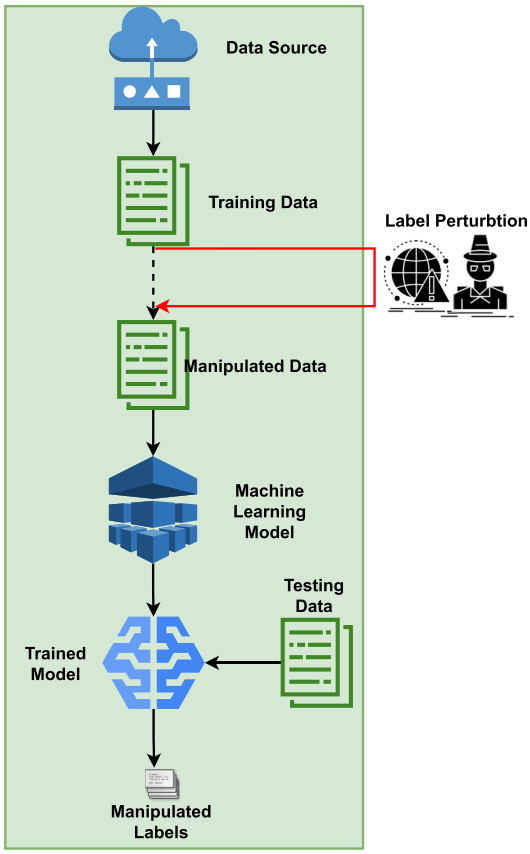


Fig. 1. Label manipulation attack on ML pipeline.

Fig. 1. Within industrial SDNs that utilize ML-based NIDS, label manipulation attacks can be detrimental as they can put the infrastructure at risk. These attacks can gradually shift the decision boundaries of the NIDS through the manipulation of training labels. Due to this gradual shifting, data samples that should be classified correctly are incorrectly predicted, compromising the performance and the predictions of the ML classifier [8]. Within the context of SDN NIDS, these attacks could misidentify attack labels as normal network traffic, which can adversely affect network functionality and resources. These attacks can also misrepresent one attack category for another, altering the network administration's response strategies, and leading to compromised systems, incorrect responses, and revenue loss. Label manipulation attacks can also be exacerbated if they attack SDN topologies that are part of critical national and industrial infrastructures.

In this paper, we propose a novel adversarial poisoning label manipulation attack that targets the SDN controller and aims to corrupt ML-based NIDS, called the Cosine Similarity Label Manipulation (CSLM) attack. The proposed technique optimally manipulates the labels within NIDS training data, using the Cosine Similarity function. For experimentation, we run this attack on an open-sourced network intrusion detection dataset. The main contributions of our proposed work include:

- Proposing a novel label manipulation attack called CSLM attack.
- Running the proposed attack on an open-sourced network intrusion detection dataset.
- Studying the impact of the proposed attack on both single and multi-controller SDN setups.

The rest of the paper is structured as follows: Section II provides the literature study. Multi-controller SDN (MSDN) setup and the system model for our research are presented in Section III, while the proposed methodology is illustrated in Section IV. Section V presents experimental results and discussions. Finally, conclusions are drawn in Section VI.

II. Related Work

Adversarial attacks against ML systems are becoming a contemporary problem that many researchers and laboratories are starting to focus on. Much of the research in this domain focuses on security problems in computer vision systems [9], speech recognition [10], spam filtering [11], and fraud detection [12] [13]. In [9], the authors proposed ShapeShifter, an adversarial attack model that is capable of creating adversarially perturbed images that make it difficult for reliable sign detection in autonomous driving applications. The work in [10] focused on ML classifiers that unconsciously reveal their statistical information and then, use this information to hack classifiers to reduce performance in applications like speech recognition. The authors in [11] experimented on how adversarial attacks can impact the security of ML-based email spam filters. The work in [12] developed one-class adversarial networks for fraud detection with only normal users in the training data. Researchers in [13] proposed a survival analysis based on an early fraud detection model that mapped dynamic user activity to survival probability, therefore guaranteeing to decrease over time.

Contemporary NIDS utilize ML and deep learning methods to provide more intelligent and sophisticated defensive solutions for protecting SDNs from cyber attacks [14] [15] [1] [2] [16]. Due to the progressive increase in their usage, research has also extended to studying the impact of adversarial attacks on ML-based NIDS. Certain adversarial attacks have been proposed in the literature that are geared towards ML-based NIDS [17] [18] [19]. In [17], the authors propose a novel adversarial attack against deep learning NIDS. The method consists of two techniques which include model extraction, to replicate the black-box model of a deep learning method, and a saliency map, to disclose the impact of each packet attribute on the detection results. The work in [18] proposed a novel hierarchical adversarial attack generation method to realize the level-aware black-box adversarial attack strategy that targets graph neural network-based NIDS. The researchers in [19] explored a DoS adversarial attack to craft adversarial samples on an artificial neural network-based NIDS.

Label manipulation in NIDS has also been an attack category that has received some recent limited attention [20], [21], [22]. The work conducted in [20] used a Generative Adversarial Network (GAN) to generate synthetic intrusion traffic to test the influence of these attacks on the accuracy of machine learning-based intrusion detection systems. Correspondingly, the authors in [21] focus on training ML-based malware detectors when

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

their training data has been poisoned using label manipulation. The limitation of both of these works is that they focus on studying the impact of poisoning through the lens of random label manipulation, which is a very simple design choice and will be easier to detect than an attack that contains a targeted design that can impact the ML-based NIDS optimally. In [22], the authors propose a targeted label manipulation/poisoning attack that aims to flip 0%-50% of labels in a training dataset for an SVM-based NIDS using the Clever Hans python library [23]. However, the experiments conducted in this work only focused on label manipulation attacks against Support Vector Machine (SVM)-based NIDS. They did not study its effects on other customarily used NIDS algorithms like Random Forest (RF) and Multi-Layer Perceptron (MLP).

In contrast to these works, our work proposes two new targeted label manipulation attacks that aim to deteriorate ML-based NIDS optimally. We also analyze the impact of the proposed attacks on both single and multi-controller SDN setups. The work in this research can catalyze to inspire new methodologies to protect ML-based NIDS from adversarial poisoning attacks.

III. Multi-Controller Setup and System Model

A. Multi-Controller SDN Architecture

SDNs are progressively increasing in usage within various organizational applications financial services, retail, healthcare, and manufacturing. It is predicted that the global SDN market value is estimated to reach $35.6 billion by 2026 [24]. As network traffic within SDNs increases, single-controller setups cannot deal with a great number of flow requests due to the limited capacity of the SDN controller. Similarly, if the single controller fails, the switches cannot properly route the new incoming packets, which adversely affects network operations. To address the above operational overload, MSDN setups have started becoming more prevalent across industrial domains. This architecture benefits the SDN topology as it effectively alleviates the flow processing pressure faced by single controllers [25].

B. System Model

We assume an industrial MSDN setup as shown in Fig. 2, consisting of three main levels:

- *Command Center:* This level houses important SDN applications that communicate with all the SDN controllers in the topology by leveraging network information like topology, state, and statistics. Examples can be applications for network automation, management, and analytics. Network administrators oversee this level to ensure the SDN is running efficiently and securely.
- *Control Plane:* This level contains all the centralized SDN controllers that regulate the data plane network devices and network policies for each respective SDN physical topology.
- *Data Plane:* This level contains all the physical individual SDN topology. These infrastructures consist of physical network devices that make up the network topology like routers, switches, bridges, and repeaters. In this plane, each physical individual topology communicates with its respective SDN controller in the Control Plane.

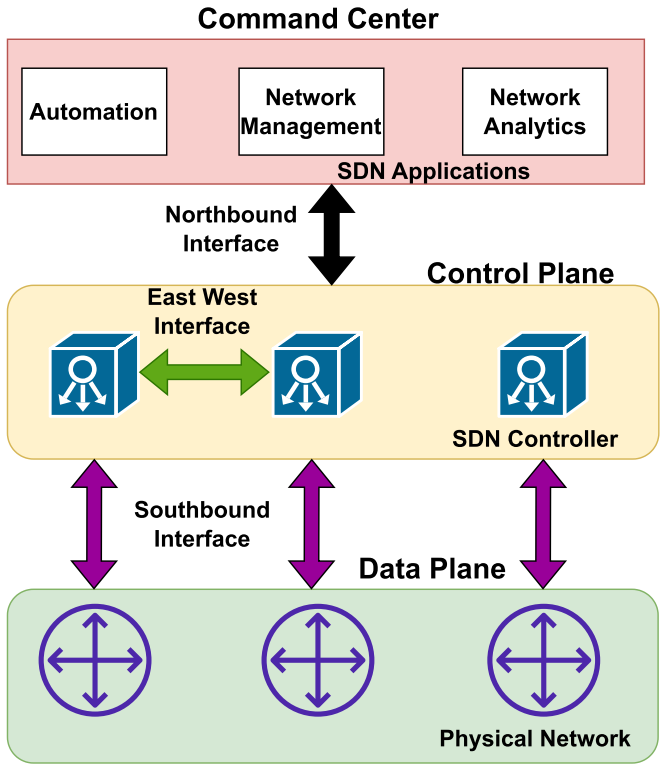


Fig. 2. Multi-controller SDN setup.

For our system model, we present the following assumption:

Assumption 1: SDN setup: Within the real-world context, every physical SDN topology in an MSDN setup can vary depending on the number of users, nodes, devices, and geographical spread of the network. In certain cases, there may be shared switches between various topologies that are under the control of multiple SDN controllers. However, for simplicity purposes, in this work, we assume that all the physical topologies are the same with the same number of parameters. We also assume that each SDN topology is administered by a unique SDN controller situated in the control plane.

IV. Methodology

A. Raw Topology and NIDS Setup

We represent the set of SDN topologies α, where $\alpha = \{\alpha_1, \alpha_2, \ldots \alpha_N\}$. Similarly, we introduce the set of SDN controllers β, where $\beta = \{\beta_1, \beta_2, \ldots ...\beta_N\}$. We assume that the total number of SDN topologies and controllers in the MSDN is N. Also, we denote the number of compromised SDN controllers as N'. The proposed CSLM will be conducted on these compromised SDN controllers. Let the target network intrusion dataset, χ, consist of ϵ total samples. We represent a single dataset sample in χ as χ_i, where $i = \{0, 1 \ldots \epsilon - 1\}$. Similarly, let the labels of the same target network intrusion dataset be denoted using γ, which also consists of ϵ total labels. We represent a single label in γ as γ_i, where $i = \{0, 1 \ldots \epsilon - 1\}$. As the total number of

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ESSENTIAL VARIABLE DEFINITIONS

Variable	Definition
α	Set of SDN Topologies
β	Set of SDN Controllers
N	Number of SDN Topologies and Controllers
N'	Number of Compromised SDN Controllers
χ	Set of Samples in Network Intrusion Detection Dataset
γ	Set of Labels in Network Intrusion Detection Dataset
ϵ	Number of Samples and Labels in χ and γ
Λ	Set of Unique Labels in γ
ζ	Total Number of Unique Labels in Λ
ν_j	Number of Samples in χ associated with each Unique Label λ_j in Λ

labels and samples in the dataset are equal, we are using ϵ to represent that value. We also denote the set of unique labels in γ as Λ, where each unique label $\lambda_j \in \Lambda, j = \{0, 1...\zeta - 1\}$. Here, ζ represents the total number of unique labels that are present in Λ. Lastly, we denote ν_j as the number of samples in χ that are associated with each unique label λ_j, where j $= \{0, 1...\zeta - 1\}$. Table I defines the important variables in this research.

B. Cosine Similarity Label Manipulation Attack

Our proposed CSLM attack is conducted by maliciously manipulating the training data before it is trained by the ML algorithm for network intrusion detection. This goal is to poison the training data that would trigger sub-optimal performance and misclassifications in the ML algorithm. The first step is to model all labels that are present in Λ. This is achieved by computing the combined sum (CS) of all the samples in the training dataset χ that contain each label λ_j, element-wise. This operation occurs by:

$$\boldsymbol{\tau_j} = \sum_{j=0}^{\nu_j} \boldsymbol{\chi_i} \mid \lambda_j, j = \{0, 1...\zeta - 1\} \tag{1}$$

where τ_j represents the CS of all dataset samples in χ that are associated with the label λ_j. Next, we compute the mean value (MV) of each label λ_j from the computed CS value. The MV of each label denotes the average magnitude for each label. This average magnitude can help to decompose each label to a single point of representation, thereby making it distinctive from the other labels. This can be beneficial to distinguish each label and can help craft more optimal adversarial poisoning attacks like our proposed CSLM attack. This is achieved by:

$$\boldsymbol{\kappa_j} = \frac{\boldsymbol{\tau_j}}{\nu_j}, j = \{0, 1...\zeta - 1\} \tag{2}$$

where κ_j refers to the MV of each label λ_j. All computed values of κ_j form a set of the MVs for all labels, denoted by K, where $\boldsymbol{\kappa_j} \in K, j = \{0, 1...\zeta - 1\}$. Once the MV of each label λ_j is computed, the label manipulation process can commence. For this, we utilize the cosine similarity measurement [26]. Cosine similarity is a measurement that calculates the similarity between two vectors of an inner product space. This is typically done by measuring the cosine of the angle between these two vectors to determine if they are pointing approximately in the same direction or not. An illustration of the functionality of the cosine similarity measurement can be seen in Fig. 3. In this figure, vector A is the target vector. Vector B has an angle of Θ_B to vector A. Similarly, vector C has an angle of Θ_C to vector A. In this scenario, we note that $\Theta_B < \Theta_C$. Hence, using Cosine similarity, we can observe that vector B is more similar to vector A than vector C. We compute the cosine similarity of each target label's MV κ_j with the remaining labels' MV $\boldsymbol{\kappa'_j}$ using:

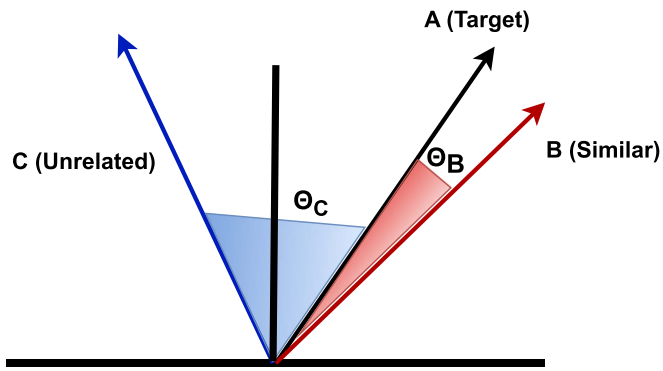


Fig. 3. Cosine similariy functionality.

$$\boldsymbol{\Theta_j} = \frac{\boldsymbol{\kappa_j} \cdot \boldsymbol{\kappa'_j}}{|\boldsymbol{\kappa_j}||\boldsymbol{\kappa'_j}|} \tag{3}$$

where Θ_j represents the cosine similarities of each target label MV κ_j with the remaining labels' MVs $\boldsymbol{\kappa'_j}$. $\boldsymbol{\kappa'_j}$ collectively represents all the other labels' MVs besides the target label's MV that exist in K. Each target label MV is compared against each of the remaining labels' MVs in K to receive a cosine similarity value for each remaining labels' MVs. Therefore, if the size of K is ζ, then $\boldsymbol{\kappa_j}$ will have $\zeta - 1$ cosine similarity values as it will compute cosine similarities with all of the other target labels' MVs. An illustration of the cosine similarity algorithm is provided in Algorithm 1. All computed values of Θ_j form a set of the cosine similarities for all labels, denoted by Θ, where $\boldsymbol{\Theta_j} \in \boldsymbol{\Theta}, j = \{0, 1...\zeta - 1\}$. The representation for the structure of the cosine similarity computation procedure in $\boldsymbol{\Theta_j}$ and $\boldsymbol{\Theta}$, as conducted in Equation 3, is illustrated in Fig. 4.

After the cosine similarity computations, we sort the magnitudes in $\boldsymbol{\Theta_j}$ from the smallest value to the largest value to create our attack vectors. For the proposed work, we put forth the following two attack types: **Maximum Cosine Similarity (Max-CSLM)** and **Minimum Cosine Similarity (Min-CSLM)**.

1) Max-CSLM: In this label manipulation attack, the target label is replaced with another dataset label that is maximally similar to it. To find this dataset label, we must select the index of the largest element in our sorted dataset $\boldsymbol{\Theta_j}$. The goal of this attack is to minimally alter the training dataset labels, to be minimally detectable. Despite being minimally detectable, this attack will still minimally compromise NIDS by triggering sub-optimal performance and misclassifications in the ML algorithm. A Max-CSLM attack can be conducted using:

$$\lambda'_j = \arg\max \boldsymbol{\Theta_j} | \lambda_j, j = \{0, 1...\zeta - 1\} \tag{4}$$

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

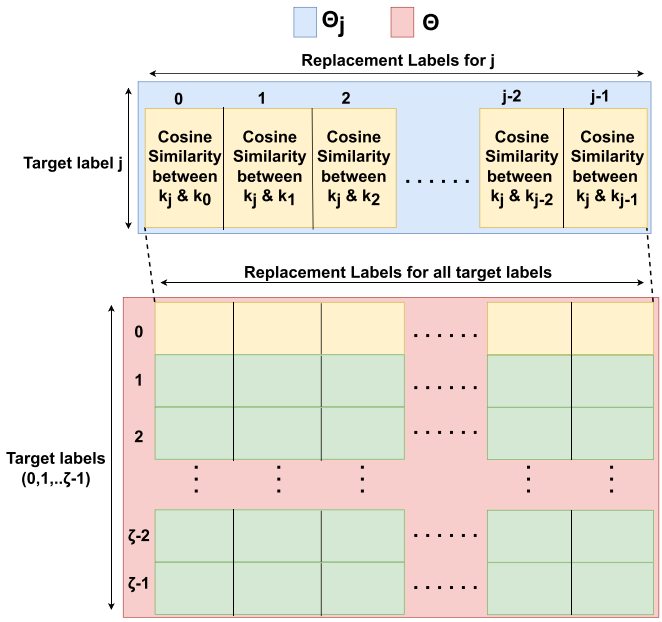


Fig. 4. Cosine similarity computation procedure in Θ_j and Θ.

Algorithm 1: Cosine Similarity Algorithm.

Input : Set of Label MVs K
Output: Matrix of cosine similarities Θ

$\Theta \leftarrow$ Empty two-dimensional matrix
foreach $i \in K$ **do**
  foreach $j \in K$ **do**
    if *i not equal to j* **then**
      Θ_{ij} = *Cosine similarity of MVs i and j;*
      Θ[i][j] = Θ_{ij}
    end
  end
end

where λ'_j represents the replacement label for the target label λ_j in the dataset χ.

2) Min-CSLM: In this label manipulation attack, the target label is replaced with another dataset label that is minimally similar to it. To find this dataset label, we must select the index of the smallest element in our sorted dataset Θ_j. The goal of this attack is to maximally alter the training dataset labels, to be maximally detectable. This attack does not prioritize detectability. Instead, it prioritizes the capability to cause maximal damage in minimal time, by compromising NIDS by triggering sub-optimal performance and misclassifications in the ML algorithm. A Min-CSLM attack can be conducted using:

$$\lambda'_j = \arg\min \Theta_j | \lambda_j, j = \{0, 1 \ldots \zeta - 1\} \tag{5}$$

where λ'_j represents the replacement label for the target label λ_j in the dataset χ. All computed values of λ'_j form a set of the malicious labels, denoted by Λ', where $\lambda'_j \in \Lambda', j = \{0, 1 \ldots \zeta\}$. Fig. 5 illustrates a visual depiction of the mechanism behind the proposed Max and Min-CSLM attacks. Additionally, we introduce a parameter Φ, that serves as a control parameter for our CSLM attack. The value of Φ, where $0 \leq \Phi \leq 1$, determines the ratio of dataset samples that will be maliciously replaced. A value of 0 means that the adversary does not manipulate any labels, while a value of 1 means that the adversary manipulates all labels in the dataset X. Lower values of Φ signify that the adversary is attempting to evade detection by the NIDS while continuing to compromise its performance and safety. Higher values of Φ denote that the adversary does not care about being detected, and wishes to inflict as much damage on the SDN as possible. Based on this, the number of data samples that will be infected in the dataset will be:

$$\eta = \lfloor \Phi \epsilon \rfloor \tag{6}$$

The final step is to randomly replace the η number of dataset samples in χ to form the malicious dataset χ' that will be used to adversarially train the ML-based NIDS for the MSDN.

C. Evaluation

For our proposed label manipulation attack, we must evaluate the impact of this adversarial threat on both the local SDN and the global MSDN. For evaluating the impact on the local SDN environment, we use Accuracy (A) and the mean scores for precision (P_μ), recall (R_μ), and f-measure (F_μ) for the multi-class classification problem, that is defined using:

$$A = \frac{TN + TP}{TN + FP + TP + FN} \tag{7}$$

$$P_\mu = \frac{\sum_0^\Lambda \frac{TP}{TP+FP} | \lambda_j}{\zeta} \tag{8}$$

$$R_\mu = \frac{\sum_0^\Lambda \frac{TP}{TP+FN} | \lambda_j}{\zeta} \tag{9}$$

$$F_\mu = \frac{\sum_0^\Lambda 2\frac{P_\mu R_\mu}{P_\mu + R_\mu} | \lambda_j}{\zeta} \tag{10}$$

where the fundamental variables include True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).

We must also evaluate the impact this type of attack can have on the MSDN scenario. For this, we employ a utility function defined by:

$$\omega_a = \zeta \Delta \tag{11}$$

Here ω_a represents the utility achieved from a single SDN a that forms a set of utilities for all SDNs, denoted by Ω, where $\omega_a \in \Omega, a = \{0, 1 \ldots N - 1\}$. Here, Δ represents the computed Matthews Correlation Coefficient (MCC) for the SDN. The equation for MCC is:

$$\Delta = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{12}$$

We chose to employ the MCC as it provides a comprehensive evaluation of an ML classifier performance incorporating all fundamental variables. It has also been actively used to evaluate ML algorithms in literature [27] [28]. The utility can physically

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

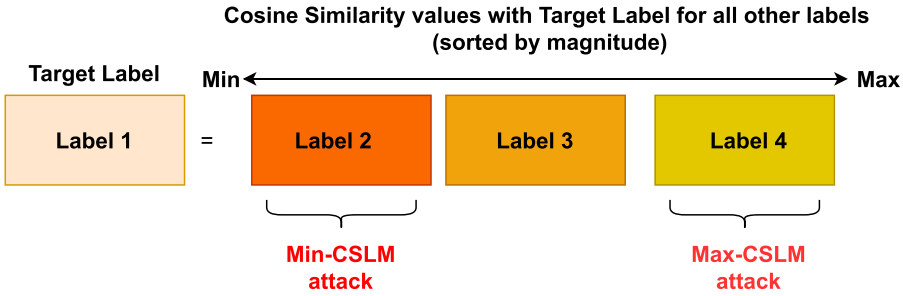


Fig. 5. Visual Depiction of Mechanism behind Max and Min-CSLM attacks.

represent an industrial organization's financial gain or reward for the proper functionality of its MSDN infrastructure. Any malicious compromise of these MSDN infrastructures could reduce the financial gain that is being achieved, leading to a loss in efficiency, productivity, and optimal functionality. Also, depending on the importance and application of this industrial MSDN, any compromise could lead to a loss in revenue, reputation, and intellectual property, and could also put the public in danger.

The final utility of the entire MSDN is computed using:

$$\omega_T = \sum_{0}^{\Omega} \omega_a \tag{13}$$

where ω_T represents the total utility achieved for the MSDN. For our proposed methodology, we present the following assumptions:

Assumption 2: Point of compromise: As the proposed CSLM attacks are targeting the training labels, it is essential for the adversary to have access to this training data collection setup. Therefore, we can assume that the adversary can achieve this malicious access by compromising integral points in the SDN architecture. Compromising high-value targets like the SDN controller can provide complete access to the training labels, while lower-value targets like a data plane switch or end hosts can provide access to partial training labels.

Assumption 3: Adversary possesses NIDS knowledge: Contemporary methods to perform NIDS primarily use ML-based methods, where training data is collected and trained offline, and a trained model is placed online. Due to this being general knowledge, an attacker can assume that the target MSDN also uses an ML-based method. Also, if they can compromise the SDN infrastructure, then the labels that are being collected as part of network intrusion detection can also be unearthed by the adversary. Other ways an adversary can identify important training labels would be through a black-box attack where the attacker can simply feed data samples through the ML model and map the labels to the inputs.

V. Experimentation, Results, and Analysis

A. Setup

For our MSDN scenario, $N = 3$, and all topologies are each connected to their unique SDN controller. We also assume $N' = 1$, upon which the CSLM attack will be conducted. All SDN controllers communicate with a centralized command center as seen in Fig. 2. For experimentation, we are using the UNR-IDD dataset [29], as it is a relevant NIDS dataset for our experiments. We are also experimenting with the impact of our CSLM attack on three ML algorithms: RF, MLP, and SVM, due to their wide usage for studying network intrusion detection in the literature. We assume $\Phi = 0.1$, as the magnitude of Φ should be low to avoid instantaneous detection, while still being able to incur a negative impact on the NIDS performance.

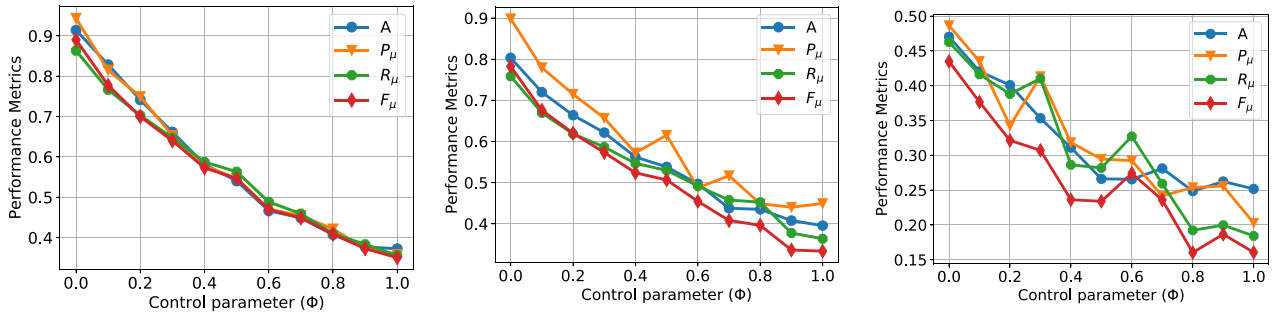
B. Experiments

We begin the experimentation by analyzing the impact of the proposed CSLM attacks on the observable performance metrics for various ML-based NIDS. Fig. 6 illustrates the impact of the Min-CSLM attack on the ML-based NIDS across RF, SVM, and MLP-based classifiers. Here, we can observe that, as the value of Φ increases, the performance decreases for the A, P_μ, R_μ, and F_μ metrics, across the three different classifiers. We observe that the RF and SVM classifiers start with decent performance but degrade to sub-optimal performance, and the MLP, which started with sub-optimal performance, degrades to low performance as the value of Φ increases. This is further reinforced in Table II, where we note that the RF undergoes the most deterioration between the highest and lowest observed A, P_μ, R_μ, and F_μ scores, followed by the SVM and the MLP.

This suggests that the Min-CSLM attacks affect the RF-based NIDS the hardest as it showcases the largest performance deterioration. This can be attributed to a random forest being an ensemble of non-robust decision trees (DT). DTs utilize a "divide and conquer" approach to generalizing a dataset which creates dependence on a subset of highly relevant dataset features to make predictions and whose performance suffers from the presence of complex interactions. Creating disruptions within the training dataset labels with a CSLM attack can make a DT perform worse, which propagates to the rest of the RF [30]. This is especially significant as the RF classifier is one of the most effective ML algorithms for NIDS in literature [31], and the proposed Min-CSLM attack seems to compromise the algorithm's performance.

Similarly, we also analyze the performance achieved by the ML-based NIDS against the Max-CSLM attack, as illustrated in Fig. 7. Here, we observe that, as the value of Φ increases,

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.



(a) RF-based NIDS performance against Min CSLM attack

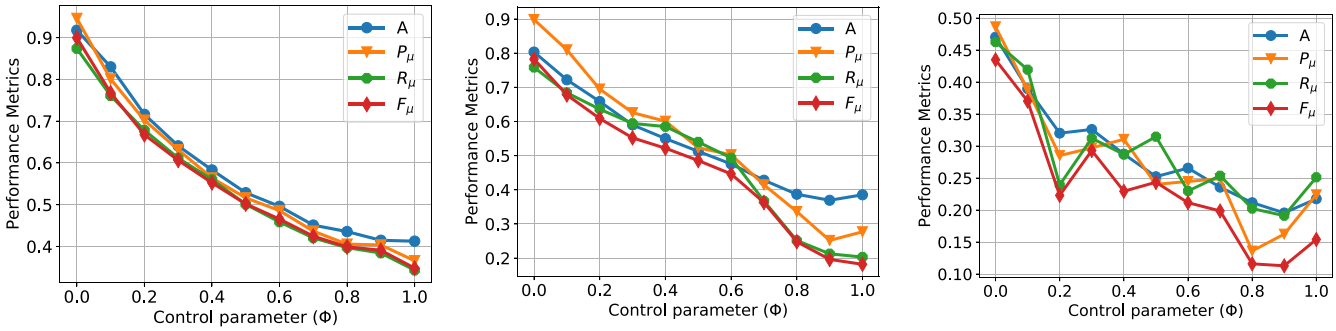
(b) SVM-based NIDS performance against Min CSLM attack

(c) MLP-based NIDS performance against Min CSLM attack

Fig. 6. Performance of ML-based NIDS against the Min CSLM attack.

TABLE II
PERFORMANCE METRIC FLUCTUATIONS UNDER MIN CSLM ATTACK

	Highest Value				Lowest Value				Deterioration			
Classifier	**A**	P_μ	R_μ	F_μ	**A**	P_μ	R_μ	F_μ	**A**	P_μ	R_μ	F_μ
RF	0.91	0.94	0.86	0.88	0.37	0.35	0.35	0.35	0.54	0.58	0.50	0.53
SVM	0.80	0.89	0.75	0.78	0.39	0.44	0.36	0.33	0.40	0.45	0.39	0.44
MLP	0.47	0.48	0.46	0.43	0.24	0.20	0.18	0.16	0.22	0.28	0.27	0.27



(a) RF-based NIDS performance against Max CSLM attack

(b) SVM-based NIDS performance against Max CSLM attack

(c) MLP-based NIDS performance against Max CSLM attack

Fig. 7. Performance of ML-based NIDS against the Max CSLM attack

TABLE III
PERFORMANCE METRIC FLUCTUATIONS UNDER MAX CSLM ATTACK

	Highest Value				Lowest Value				Deterioration			
Classifier	**A**	P_μ	R_μ	F_μ	**A**	P_μ	R_μ	F_μ	**A**	P_μ	R_μ	F_μ
RF	0.91	0.94	0.87	0.89	0.41	0.36	0.34	0.34	0.50	0.58	0.53	0.55
SVM	0.80	0.89	0.75	0.78	0.36	0.25	0.20	0.18	0.43	0.64	0.55	0.60
MLP	0.47	0.48	0.46	0.43	0.19	0.13	0.19	0.11	0.27	0.35	0.27	0.32

the performance for the A, P_μ, R_μ, and F_μ metrics, across the three different classifiers. Like the Min-CSLM attack, the RF and SVM classifiers start with decent performance but degrade to sub-optimal performance, and the MLP, which started with sub-optimal performance, degrades to low performance as the value of Φ increases. This is further emphasized in Table III, where we note that the SVM undergoes the most deterioration between the highest and lowest observed P_μ, R_μ, and F_μ scores, followed by the RF and MLP.

This suggests that the Max-CSLM attacks affect the SVM-based NIDS the hardest as it showcases the largest performance deterioration. When training labels are linearly separable, SVM hyperplanes can be computed quickly and efficiently. However, when they are not, the data usually is mapped with a kernel function into a higher dimensional space where a hyperplane can be fitted to separate the labels. Inconsistent and manipulated training labels can adversely affect the functionality and performance of an SVM. This is especially significant as the

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

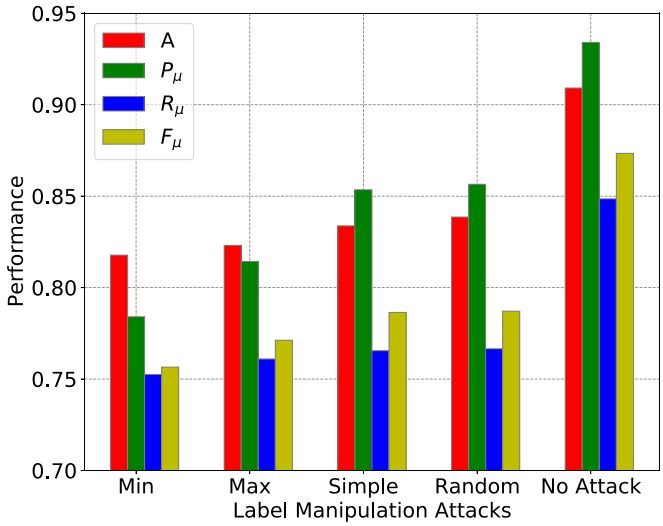


Fig. 8. Performance of Max and Min CSLM against other label manipulation attacks.

SVM classifier is one of the most effective ML algorithms for NIDS in literature [32], and the proposed Max-CSLM attack seems to compromise the algorithm's performance.

Next, we observe the performance of the proposed CSLM attacks compared to two other intuitive label manipulation attacks:

- *Simple Attack:* All labels in the training data are maliciously altered to a single label.
- *Random Attack:* All labels in the training data are maliciously altered to a randomly chosen label.

These label manipulation attacks serve as benchmarks only, upon which we evaluate the effectiveness of our proposed CSLM attacks. Fig. 8 illustrates the performance achieved by the ML-based NIDS when the label manipulation attacks are performed. We observe that a Random Label Manipulation attack provides the least deterioration in classifier performance, followed by the Simple Label Manipulation attack. This can be attributed to the fact that a simple attack changes all labels to a single label, making it more difficult to differentiate between various classifications, whereas a training dataset that has undergone a Random Label Manipulation will have a variable number of labels in it. Out of the proposed CSLM attacks, the Min-CSLM provides more deterioration than the Max-CSLM attack as the attack is designed to maximally inflict damage on the dataset by not prioritizing detectability. In contrast, the Max-CSLM attack will provide less deterioration but will be harder to detect, due to minimally altering the training labels, and hence, being minimally detectable. Additionally, these two proposed techniques perform better than the other two benchmark techniques as the labels are optimally selected to incur damage, whereas both Simple and Random Label Manipulation linearly and randomly manipulate the training labels, respectively.

Next, we focus on the impact of our proposed CSLM attacks on the entire MSDN. We evaluate this scenario under two conditions:

- *Uniform:* All the SDN controllers are performing network intrusion detection using the same ML algorithm.
- *Variable:* Each SDN controller is using a different ML algorithm to perform network intrusion detection.

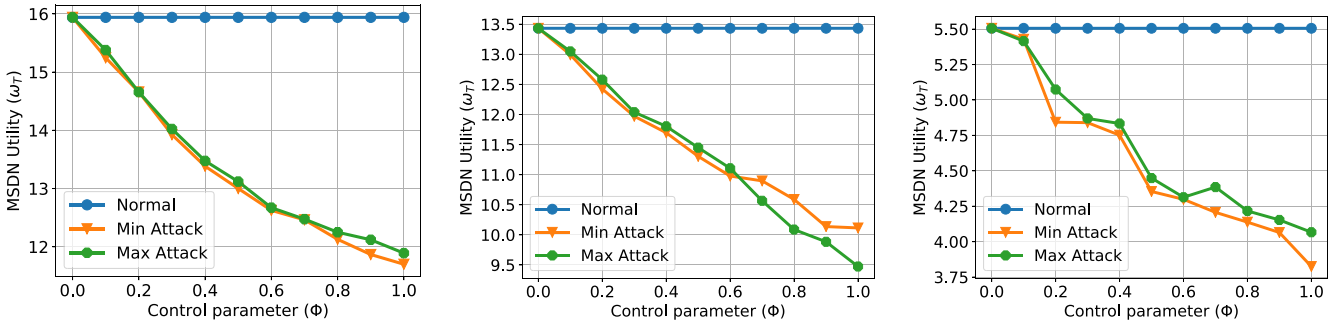
First, we observe the impact of N' on a uniform MSDN setup and the achieved utility ω_T, illustrated in Fig. 9 . We note that, in all three situations, under no attack, the achieved utilities by the MSDNs stay the same as the value of Φ increases. However, when facing Min and Max-CSLM attacks, the utilities of the MSDNs progressively decrease as Φ increases. We note that as the value of Φ increases, the achieved utility of RF and MLP-based uniform MSDNs remains higher under a Max-CSLM attack than a Min-CSLM attack. This re-emphasizes the designs of the two attack types. The Min-CSLM provides lower utility than the Max-CSLM attack as the attack is designed to maximally inflict damage on the dataset by not prioritizing detectability. In contrast, the Max-CSLM attack will provide better utility but will be harder to detect, due to minimally altering the training labels, and hence, being minimally detectable. An adversary wanting to compromise an RF or MLP-based uniform MSDN should use a Max-CSLM attack to incur minimal damage by prioritizing undetectability and use a Max-CSLM attack to incur maximal damage by not prioritizing undetectability.

Here, we notice that for the SVM-based uniform MSDN, the Max-CSLM attack achieves more utility than the Min-CSLM attack at $\Phi < 0.65$, after which the utilities switch. This phenomenon can be attributed to the fact that SVMs can generate hyperplanes efficiently when the labels are linearly separable but need the assistance of a kernel function when they are not. An attack that manipulates training labels could adversely affect the functionality and performance of an SVM-based uniform MSDN. Hence, an adversary wanting to attack an SVM-based uniform MSDN should aim to use a Min-CSLM attack if they wish to manipulate $< 65\%$ of training labels and use a Max-CSLM attack if they wish to corrupt $\geq 65\%$ of training labels.

In Table IV, we observe the utility decrease percentage achieved by each ML algorithm when facing the proposed CSLM attacks. We note that uniform MSDNs that consist of SVM incur the most utility decrease when faced with a Max-CSLM attack. This observation is consistent with the previously observed deductions which note the vulnerabilities of the SVM algorithm to the proposed CSLM attacks. Also, we note that the MLP-based uniform MSDN incurred the most utility decrease when faced with a Min-CSLM attack. Like the SVM, the MLP is unable to accurately classify labels that are not linearly separable. This is because, unlike a neural network, an MLP does not utilize a non-linearity function to differentiate between non-linear labels [33]. An adversarial attack that exploits training labels could adversely affect the functionality and performance of an MLP-based uniform MSDN. Finally, we observe the impact of a compromised SDN controller N' on a uniform MSDN setup and the utility that is being achieved. Fig. 10 illustrates the observed utilities for a variable MSDN setup. To minimize any variability, we have ensured that in each scenario, a separate ML-based NIDS is being used for the compromised controller N'.

In Fig. 10(a), (b), and (c), $N' =$ RF, SVM, and MLP, respectively, and we refer to them as RF-based, SVM-based, and MLP-based variable MSDNs, separately. From the figure, we

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.



(a) Observed Utility of Uniform MSDN when all NIDS are RF

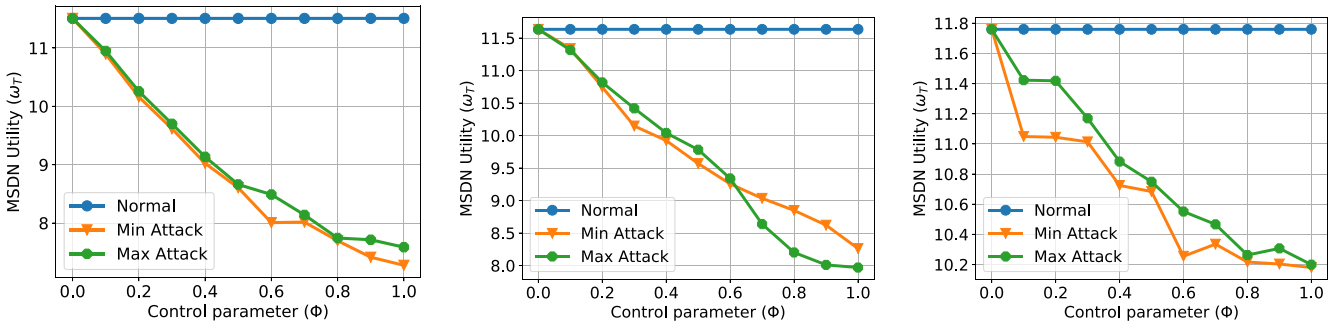
(b) Observed Utility of Uniform MSDN when all NIDS are SVM

(c) Observed Utility of Uniform MSDN when all NIDS are MLP

Fig. 9. Observed utilities of a uniform MSDN setup.

TABLE IV
UNIFORM MSDN UTILITY DECREASE UNDER CSLM ATTACKS

Classifier	Max-CSLM Attack		Min-CSLM Attack		Utility Decrease	
	Highest Utility Value	Lowest Utility Value	Highest Utility Value	Lowest Utility Value	Max-CSLM	Min-CSLM
RF	15.93	11.89	15.93	11.69	25.40%	26.59%
MLP	5.50	4.06	5.50	3.82	26.14%	30.53%
SVM	13.43	9.47	13.43	10.11	29.47	24.70%



(a) Observed Utility of Variable MSDN when N' = RF

(b) Observed Utility of Variable MSDN when N' = SVM

(c) Observed Utility of Variable MSDN when N' = MLP

Fig. 10. Observed utilities of a variable MSDN setup.

can see that, in all three situations, under no attack conditions, the achieved utilities by the MSDN stay the same as the value of Φ increases. However, when facing Min and Max-CSLM attacks, the utilities of the MSDN progressively decrease as Φ increases. We note that as the value of Φ increases, the achieved utilities of RF and MLP-based variable MSDNs remain higher under a Max-CSLM attack than a Min-CSLM attack, reinforcing the mechanism design behind both attack categories. The Max-CSLM attack aims at minimal variability during label manipulation, making it more difficult to detect. In contrast, the Min-CSLM attack aims to inflict maximal damage and does not care about detectability. An adversary wanting to compromise an RF or MLP-based variable MSDN should use a Max-CSLM attack to incur minimal damage by prioritizing undetectability and use a Max-CSLM attack to incur maximal damage by not prioritizing undetectability.

We note that for the SVM-based variable MSDN, the Max-CSLM attack achieves more utility than the Min-CSLM attack at $\Phi < 0.65$, after which the utilities switch. This is similar to the observations made in Fig. 9(b) and can be attested to SVMs being capable of generating hyperplanes efficiently when the labels are linearly separable, but needing the assistance of a kernel function when they are not. An attack that manipulates training labels could adversely affect the functionality and performance of an SVM-based variable MSDN. Hence, an adversary wanting to attack an SVM-based variable MSDN should aim to use a Min-CSLM attack if they wish to manipulate $< 65\%$ of training labels and use a Max-CSLM attack if they wish to corrupt $\geq 65\%$ of training labels.

In Table V, we observe the utility decrease percentage achieved by each ML algorithm when facing the proposed CSLM attacks. Here, we notice that the variable MSDN where

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

TABLE V
VARIABLE MSDN UTILITY DECREASE UNDER CSLM ATTACKS

	Max-CSLM Attack		Min-CSLM Attack		Utility Decrease	
Classifier	**Highest Utility Value**	**Lowest Utility Value**	**Highest Utility Value**	**Lowest Utility Value**	**Max-CSLM**	**Min-CSLM**
RF	11.50	7.58	11.50	7.27	34.07%	36.77%
MLP	11.75	10.19	11.75	10.18	13.26%	13.43%
SVM	11.63	7.96	11.63	8.26	31.50%	28.99%

$N' =$ RF incurred the most utility decrease out of all variable MSDN setups for both Max-CSLM and Min-CSLM attacks. This can be a consequence of a random forest being an ensemble of DTs, which are non-robust classifiers. Due to the "divide and conquer" approach, many DTs create a dependence on a subset of highly relevant dataset features to make predictions and whose performance suffers from the presence of complex interactions. Adversarial attacks that negatively influence training labels with a CSLM attack can deteriorate a DT performance, making the performance of the RF suffer.

Defending against adversarial attacks like the proposed CSLM attacks is a non-trivial task. Current research methods are focused on generating strategies to protect ML pipelines from adversarial attacks. However, these attacks are difficult to defend against due to their multi-variate computational complexity and non-linear and non-convex nature of perturbation, and the operational requirements of most ML algorithms [34]. Also, most defensive solutions that are being proposed are isolated to certain algorithm categories and applications as they utilize the inherent features of these methods and other application characteristics. Proposed defensive techniques like adversarial training [35] prove to be inefficient against the proposed CSLM attacks due to the unknown nature of the perturbations that must be predicted to train the ML model, while other techniques like gradient hiding [36] are only geared towards gradient-based ML algorithms like neural networks. Techniques such as feature squeezing [37] can also prove to be ineffectual as these methods reduce the feature surface but will be unsuccessful against label manipulation attacks like the proposed CSLM attacks [38]. To the best of our knowledge, there is no standard defense mechanism that has been proposed in the literature that can adequately protect against adversarial poisoning attacks. Specifically, in network security and in ML-based NIDS where these attacks have been analytically presented to have disastrous consequences.

VI. CONCLUSION AND FUTURE WORK

In this paper, we propose a new type of adversarial label manipulation attack called Cosine Similarity Label Manipulation (CSLM). This attack primarily focuses on compromising the predictions and performance of ML-based NIDS that protect SDNs. We propose two versions of the attack called the Max and Min-CSLM attacks that employ an optimized cosine similarity function to perform the attack, that focuses on undetectability and utility, respectively. We evaluated the efficacy of these attacks on a single SDN controller topology to analyze the impact on the observed performance metrics. Also, we compare the impact of these proposed attacks to other customary label manipulation attacks. Additionally, we note the impact of these proposed attacks on MSDN setups. Results show that the proposed CSLM attacks provide substantial deterioration of ML-based NIDS classifier performance in single SDN setups, specifically those that utilize RF and SVMs. We also note that RF, SVM, and MLP classifiers are extensively vulnerable to these attacks in MSDN setups as they incur the most observed utility deterioration. Future work in this line of research includes further optimizing these proposed attacks to make them more lethal against ML-based NIDS. Other research directions include developing defensive schemes to protect ML-based NIDS from these types of adversarial attacks. Additionally, we would like to take the results from this research and develop features that can create more robust ML pipelines that are capable of being resilient even when data and/or label manipulations have occurred in the realm of network security.

REFERENCES

[1] T. Das, R. M. Shukla, and S. Sengupta, "The devil is in the details: Confident & explainable anomaly detector for software-defined networks," in *Proc. IEEE 20th Int. Symp. Netw. Comput. Appl.*, 2021, pp. 1–5.
[2] T. Das, O. A. Hamdan, S. Sengupta, and E. Arslan, "Flood control: TCP-syn flood detection for software-defined networks using openflow port statistics," in *Proc. 2022 IEEE Int. Conf. Cyber Secur. Resilience*, 2022, pp. 1–8.
[3] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. 2015 Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.
[4] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep learning approach for network intrusion detection in software defined networking," in *Proc. 2016 Int. Conf. Wireless Netw. Mobile Commun.*, 2016, pp. 258–263.
[5] M. Ozay, I. Esnaola, F. T. Y. Vural, S. R. Kulkarni, and H. V. Poor, "Machine learning methods for attack detection in the smart grid," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 8, pp. 1773–1786, Aug. 2016.
[6] S. Tan, J. M. Guerrero, P. Xie, R. Han, and J. C. Vasquez, "Brief survey on attack detection method for cyber-physical systems," *IEEE Syst. J.*, vol. 14, no. 4, pp. 5329–5339, Dec. 2020, doi: 10.1109/JSYST.2020.2991258.
[7] C. Wang, J. Chen, Y. Yang, X. Ma, and J. Liu, "Poisoning attacks and countermeasures in intelligent networks: Status QUO and prospects," *Digit. Commun. Netw.*, vol. 8, no. 2, pp. 225–234, 2022.
[8] M. Kloft and P. Laskov, "Online anomaly detection under adversarial impact," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 405–412.
[9] S.-T. Chen, C. Cornelius, J. Martin, and D. H. Chau, "Shapeshifter: Robust physical adversarial attack on faster R-CNN object detector," in *Proc. Mach. Learn. Knowl. Discov. Databases: Eur. Conf.*, Dublin, Ireland, 2019, pp. 52–68.
[10] G. Ateniese, L. V. Mancini, A. Spognardi, A. Villani, D. Vitali, and G. Felici, "Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers," *Int. J. Secur. Netw.*, vol. 10, no. 3, pp. 137–150, 2015.
[11] B. Kuchipudi, R. T. Nannapaneni, and Q. Liao, "Adversarial machine learning for spam filters," in *Proc. 15th Int. Conf. Availability, Rel. Secur.*, 2020, pp. 1–6.
[12] P. Zheng, S. Yuan, X. Wu, J. Li, and A. Lu, "One-class adversarial nets for fraud detection," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, pp. 1286–1293.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.

[13] P. Zheng, S. Yuan, and X. Wu, "Safe: A neural survival analysis model for fraud early detection," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 01, pp. 1278–1285.
[14] M. R. Hadi and A. S. Mohammed, "A novel approach to network intrusion detection system using deep learning for SDN: Futuristic approach," 2022, *arXiv:2208.02094*.
[15] A. T. Phu et al., "Defending SDN against packet injection attacks using deep learning," *Comput. Netw.*, vol. 234, 2023, Art. no. 109935.
[16] A. O. Alzahrani and M. J. Alenazi, "ML-IDSDN: Machine learning based intrusion detection system for software-defined network," *Concurrency Comput.: Pract. Experience*, vol. 35, no. 1, 2023, Art. no. e7438.
[17] H. Qiu, T. Dong, T. Zhang, J. Lu, G. Memmi, and M. Qiu, "Adversarial attacks against network intrusion detection in IoT systems," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10327–10335, Jul. 2021.
[18] X. Zhou et al., "Hierarchical adversarial attacks against graph neural network based IoT network intrusion detection system," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9310–9319, Jun. 2022.
[19] X. Peng, W. Huang, and Z. Shi, "Adversarial attack against DoS intrusion detection: An improved boundary-based method," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell.*, 2019, pp. 1288–1295.
[20] E. Alshahrani, D. Alghazzawi, R. Alotaibi, and O. Rabie, "Adversarial attacks against supervised machine learning based network intrusion detection systems," *PLoS One*, vol. 17, no. 10, 2022, Art. no. e0275971.
[21] K. Aryal, M. Gupta, and M. Abdelsalam, "Analysis of label-flip poisoning attack on machine learning based malware detector," in *Proc. 2022 IEEE Int. Conf. Big Data*, 2022, pp. 4236–4245.
[22] P. Papadopoulos, O. Thornewill von Essen, N. Pitropakis, C. Chrysoulas, A. Mylonas, and W. J. Buchanan, "Launching adversarial attacks against network intrusion detection systems for IoT," *J. Cybersecurity Privacy*, vol. 1, no. 2, pp. 252–273, 2021.
[23] N. Papernot et al., "Cleverhans v2. 0.0: An adversarial machine learning library," 2016, *arXiv:1610.00768*.
[24] "The Software-Defined Networking (SDN) market in 2022," Apr. 2022. [Online]. Available: https://www.enterprisestorageforum.com/networking/software-defined-networking-market/
[25] T. Hu, Z. Guo, P. Yi, T. Baker, and J. Lan, "Multi-controller based software-defined networking: A survey," *IEEE Access*, vol. 6, pp. 15980–15996, 2018.
[26] J. Han et al., "Getting to know your data," in *Data Mining*, vol. 2. Boston, MA, USA: Morgan Kaufmann, 2012, pp. 39–82.
[27] C. Halimu, A. Kasem, and S. S. Newaz, "Empirical comparison of area under ROC curve (AUC) and mathew correlation coefficient (MCC) for evaluating machine learning algorithms on imbalanced datasets for binary classification," in *Proc. 3rd Int. Conf. Mach. Learn. Soft Comput.*, 2019, pp. 1–6.
[28] D. Chicco, M. J. Warrens, and G. Jurman, "The Matthews correlation coefficient (MCC) is more informative than Cohen's Kappa and Brier score in binary classification assessment," *IEEE Access*, vol. 9, pp. 78368–78381, 2021.
[29] T. Das, O. Abu Hamdan, R. Shukla, S. Sengupta, and E. Arslan, "UNR-IDD: Intrusion detection dataset using network port statistics," in *Proc. IEEE 20th Consum. Commun. Netw. Conf.*, 2022, pp. 497–500.
[30] L. Rokach and O. Maimon, "Decision trees," in *Data Mining and Knowledge Discovery Handbook*. Berlin, Germany: Springer, 2005, pp. 165–192.
[31] G. Apruzzese, M. Andreolini, M. Colajanni, and M. Marchetti, "Hardening random forest cyber detectors against adversarial attacks," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 4, pp. 427–439, Aug. 2020.
[32] M. Hosseinzadeh, A. M. Rahmani, B. Vo, M. Bidaki, M. Masdari, and M. Zangakani, "Improving security using SVM-based anomaly detection: Issues and challenges," *Soft Comput.*, vol. 25, no. 4, pp. 3195–3223, 2021.
[33] M. D. Mohanty and M. N. Mohanty, "Verbal sentiment analysis and detection using recurrent neural network," in *Advanced Data Mining Tools and Methods for Social Computing*. New York, NY, USA: Elsevier, 2022, pp. 85–106.
[34] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay, "Adversarial attacks and defences: A survey," 2018, *arXiv:1810.00069*.
[35] A. Shafahi et al., "Adversarial training for free!," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 3358–3369.
[36] H. Zhang, H. Wang, Y. Cao, C. Shen, and Y. Li, "Robust data hiding using inverse gradient attention," 2020, *arXiv:2011.10850*.
[37] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," 2017, *arXiv:1704.01155*.
[38] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," 2017, *arXiv:1705.07204*.



Tapadhir Das (Member, IEEE) received the B.S. degree in computer engineering technology from the Oregon Institute of Technology, Klamath Falls, OR, USA, in 2018, and the M.S. and Ph.D. degrees in computer science and engineering from the University of Nevada, Reno, NV, USA, in 2020 and 2023, respectively. He is currently an Assistant Professor with the Department of Computer Science, University of the Pacific, Stockton, CA, USA. He is also on the Program Committee of several IEEE Conferences and a Reviewer for several high-impact journals. His research interests include smart grid security, resilient and trustworthy AI, and intelligent vehicular security.



Raj Mani Shukla received the B.Tech. degree from the Bundelkhand Institute of Engineering and Technology, Jhansi, Uttar Pradesh, India, in 2011, the master's degree from NIT Kurukshetra, Kurukshetra, Haryana, India, in 2013, and the Ph.D. degree in computer science and engineering from the University of Nevada, Reno, NV, USA, under the supervision of Dr. Shamik Sengupta. He was a Junior Research Fellow (JRF) with IIT Kanpur, Kanpur, India. He has experience in data science, machine learning, natural language processing, cybersecurity, and IoT. Previously, and was also a KTP Associate with the University of Bristol, Bristol, U.K. He is currently a Lecturer with Anglia Ruskin University, Cambridge, U.K.



Shamik Sengupta (Senior Member, IEEE) is currently a Professor with the Department of Computer Science and Engineering and the Executive Director with Cybersecurity Center, University of Nevada (UNR), Reno, NV, USA. He has authored more than 150 international conferences and journal publications. His research interests include cognitive radio and DSA networks, game theory, cybersecurity, network economics, and self-configuring wireless mesh networks. He was the recipient of an IEEE GLOBECOM 2008 Best Paper Award, International Symposium on Performance Evaluation of Computer and Telecommunication Systems 2017 Best Paper Award, NSF CAREER Award in 2012, and the UNR CSE Best Researcher Award in 2015–2016 and 2017–2018. He is also on the Organizing and Technical Program Committee of several IEEE conferences, and is on the Editorial Boards of several journals.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 06,2026 at 11:07:14 UTC from IEEE Xplore. Restrictions apply.